

NOZIONI DI BASE

INDICE SEZIONE

- ☐ PROBLEMA COMPUTAZIONALE
 - ESEMPI DI PROBLEMI COMPUTAZIONALI
- ☐ ALGORITMO E MODELLO DI CALCOLO
- ☐ PSEUDOCODICE
 - ES TRASPOSTA DI UNA MATRICE $N \times N$ DI INTERI
- ☐ STRUTTURE DATI
- ☐ ANALISI DEGLI ALGORITMI
 - TAGLIA DI UNA ISTANZA
 - COMPLESSITA' IN TEMPO
 - ANALISI COMPLESSITA' IN PRATICA
 - TERMINOLOGIA
 - ESEMPIO CRITTOGRAFIA (SCELTA DELLA TAGLIA)
 - CAVEAT SULL'ANALISI ASINTOTICA AL CASO PESSIMO
- ☐ CORRETTEZZA
 - ANALISI DI CICLI
- ☐ RICORSIONE
 - ESECUZIONE DI UN ALGORITMO RICORSIVO
 - COMPLESSITA' DI ALGORITMI RICORSIVI
 - CORRETTEZZA DEGLI ALGORITMI RICORSIVI
 - CALCOLO EFFICIENTE DI POTENZE

PROBLEMA COMPUTAZIONALE

Un *problema computazionale* è costituito da:

- Un insieme $\mathcal{I}$ di **ISTANZE** (cioè i possibili *input*).
- Un insieme $\mathcal{S}$ di **SOLUZIONI** (cioè i possibili *output*).
- Una relazione Π che ad ogni istanza $i \in \mathcal{I}$ associa *una o più* soluzioni $s \in \mathcal{S}$.

NOTA

Π è un sottoinsieme del prodotto cartesiano $\mathcal{I} \times \mathcal{S}$.

ESEMPI DI PROBLEMI COMPUTAZIONALI

ORDINAMENTO DI ARRAY DI INTERI

- $\mathcal{I} = \{A : A = \text{array di interi}\};$
- $\mathcal{S} = \{B : B = \text{array ordinati di interi}\};$
- $\Pi = \{(A, B) : A \in \mathcal{I}, B \in \mathcal{S}, B \text{ contiene gli stessi interi di } A\}.$

Ad esempio:

- $(\langle 43, 16, 75, 2 \rangle, \langle 2, 16, 43, 75 \rangle) \in \Pi.$
- $(\langle 7, 1, 7, 3, 3, 5 \rangle, \langle 1, 2, 3, 5, 7, 7 \rangle) \notin \Pi.$

VERIFICA SU DUE INSIEMI

PROBLEMA: verifica se due insiemi finiti di oggetti da un universo U sono disgiunti oppure no.

Quindi abbiamo:

- $\mathcal{I} = \{(A, B) : A, B \subset U, A, B \text{ disgiunti}\}.$
- $\mathcal{S} = \{\text{true}, \text{false}\}.$
- $\Pi = \{((A, B), S) : S = \text{true se } A \cap B = \emptyset, S = \text{false se } A \cap B \neq \emptyset\}.$

ALGORITMO E MODELLO DI CALCOLO

ALGORITMO

Procedura computazionale ben definita che trasforma un dato *input* in un *output* eseguendo una *sequenza finita* di *operazioni elementari*.

L'algoritmo fa riferimento ad un *modello di calcolo*, ovvero un'astrazione di computer che definisce l'insieme delle operazioni elementari.

Nel modello di calcolo **RAM** (*Random Access Machine*), *input*, *output*, *dati intermedi* e *programma* si trovano in *memoria* e le **operazioni elementari** sono: *assegnamento*, *operazioni logiche*, *operazioni aritmetiche*, *indicizzazione di array*, *return di un valore da una funzione*, *etc...* .

Un algoritmo A risolve un problema computazionale $\Pi \subseteq \mathcal{I} \times \mathcal{S}$ se:

1. Calcola una funzione da $\mathcal{I}$ a $\mathcal{S}$, ovvero riceve in *input* istanze $i \in \mathcal{I}$ e produce come *output* soluzioni $s \in \mathcal{S}$.
2. i ed s sono tali per cui $(i, s) \in \Pi$.

⚠ ATTENZIONE

Se Π associa *più soluzioni* ad una istanza i , per tale istanza A calcola una sola soluzione. Quale viene calcolata dipende da come viene progettato l'algoritmo.

PSEUDOCODICE

Per *semplicità* e facilità di analisi, descriviamo un algoritmo utilizzando uno *pseudocodice* strutturato come segue:

- **Algoritmo**: nome (parametri).
- **Input**: breve descrizione dell'istanza di input.
- **Output**: breve descrizione della soluzione restituita in output.

Descrizione *chiara* dell'algoritmo tramite costrutti di linguaggi di programmazione e, se utile ai fini della chiarezza, anche tramite *linguaggio naturale*, dalla quale sia facilmente determinabile la *sequenza di operazioni elementari* eseguita per ogni dato input.

ES: TRASPOSTA DI UNA MATRICE $N \times N$ DI INTERI

L'insieme delle *istanze* è:

$$\mathcal{I} = \{A : \text{matrice } n \times n \text{ di interi}\}$$

L'insieme delle *soluzioni* è:

$$\mathcal{S} = \{B : \text{matrice } n \times n \text{ di interi}\}$$

La *relazione* Π , invece, è:

$$\Pi = \{(A, B) : A \in \mathcal{I}, B \in \mathcal{S}, B = A^t\}$$

IDEA: scambiare ciascuna entry a_{ij} del *tringolo superiore* ($i = 0, \dots, n-2$ e $j > i$) con l'entry a_{ji} .

- **Algoritmo**: *TRANSPOSE*.
- **Input**: matrice A $n \times n$ di interi a_{ij} con $0 \leq i, j \leq n$
- **Output**: matrice A^t .

Operazioni:

Algorithm TRANSPOSE

```
1: for  $j \leftarrow 0$ , to  $n - 2$  do
2:   for  $j \leftarrow i + 1$  to  $n - 1$  do
3:     exchange  $a_{ij}$  with  $a_{ji}$ 
4:   end for
5: end for
```

STRUTTURE DATI

Sono usate dagli algoritmi per organizzare e accedere in modo sistematico ai dati di input e ai dati generati durante l'esecuzione.

STRUTTURA DATI

Una struttura dati è una **collezione di oggetti** corredata di *metodi di accesso* e/o *modifica*.

Presenta due *livelli di astrazione*:

- Livello **logico**: specifica l'organizzazione logica degli oggetti della collezione e la relazione input-output di ciascun metodo (a questo livello si parla di *Abstract Data Type* o *ADT*).
- Livello **fisico**: specifica il layout fisico dei dati e la realizzazione dei metodi tramite algoritmi.

Esempio in Java:

- Livello *logico*: **interfacce**.
- Livello *fisico*: **classi**.

ANALISI DEGLI ALGORITMI

L'analisi di un algoritmo A mira a studiarne l'**efficienza** e l'**efficacia**.

In particolare, essa valuta:

- **COMPLESSITA'**:
 - *Temporale*.
 - *Spaziale*.
- **CORRETTEZZA**:
 - *Terminazione*.
 - *Soluzione al problema*.

Daremo particolare attenzione alla *complessità temporale* e alla *correttezza* della soluzione al problema computazionale.

TAGLIA DI UNA ISTANZA

L'analisi di un algoritmo viene solitamente fatta partizionando le istanze in gruppi in base alla loro *taglia (size)*, in modo che le istanze di un gruppo siano tra loro *confrontabili*.

La taglia di una istanza è espressa da *uno o più valori* che ne caratterizzano la grandezza.

ESEMPI:

- *Array*: la sua lunghezza n .
- *Matrice quadrata*: numero di righe (o colonne).
- *Matrice (non quadrata)*: numero di righe e numero di colonne.

COMPLESSITA' IN TEMPO

L'obiettivo è quello di stimare il *tempo di esecuzione (running time)* di un algoritmo al fine di valutarne l'efficienza e poterlo confrontare con altri algoritmi per lo stesso problema.

Tale complessità deve essere *derivabile dallo pseudocodice*.

Un esempio di approccio che soddisfa i requisiti è l'analisi al *caso pessimo (worst-case)* in funzione dell'*istanza*:

- Conteggio operazioni elementari nel modello RAM.
- Analisi asintotica (per semplificare il conteggio).

NOTA

Esistono anche le analisi al *caso medio* e l'*analisi probabilistica*.

Sia A un algoritmo che risolve il problema Π .

COMPLESSITA' (IN TEMPO)

La *complessità (in tempo)* al *caso pessimo* di A è una funzione $t_A(n)$ definita come il *massimo numero di operazioni* elementari che A *esegue per risolvere* una

istanza di taglia n .

Tuttavia determinare $t_A(n)$ per ogni n è arduo, se non impossibile, perchè:

- E' difficile identificare l'*istanza peggiore* di taglia n .
- E' difficile contare il *massimo numero di operazioni* richieste per risolvere tale istanza, il che richiederebbe anche una specifica dettagliata del set di operazioni elementari del modello di calcolo.

Fortunatamente, *non è necessario* determinare *esattamente* $t_A(n)$, per le seguenti ragioni:

- Il *tempo di esecuzione*, che la complessità vuole stimare, dipende da tanti fattori ed è pertanto *impossibile* da quantificare in modo preciso.
- Le diverse operazioni elementari del modello RAM possono avere *impatto diverso* sui tempi di esecuzione a seconda dell'*architettura dell'elaboratore*.

Per questi motivi, ci accontentiamo di **limiti superiori ed inferiori** a $t_A(n)$, ottenuti ricorrendo all'**analisi asintotica**, cioè ignorando:

- *Fattori moltiplicativi* costanti.
- *Termini additivi* non dominanti.

RICHIAMO DI ANALISI: ORDINI DI GRANDEZZA

Per l'analisi asintotica ci vengono in aiuto gli *ordini di grandezza*.

Date due funzioni $f(n), g(n) : \mathbb{R} \rightarrow \mathbb{R}^+ \cup \{0\}$ ricordiamo la definizione delle notazioni seguenti.

O-GRANDE

O-grande

$f(n) \in O(g(n))$ se $\exists c > 0$ ed $\exists n_0 \geq 1$, costanti rispetto ad n , tali che valga:

$$f(n) \leq cg(n), \forall n \geq n_0$$

ESEMPI

- $f(n) = 3n + 4$ è $O(n)$ perchè $f(n) \leq 4n, \forall n \geq 4$.
- $f(n) = n + 2n^2$ è $O(n^2)$ perchè $f(n) \leq 3n^2, \forall n \geq 1$.
- $f(n) = c_1n + c_2$ con $c_1, c_2 > 0$ perchè $f(n) \leq (c_1 + c_2)n, \forall n \geq 1$.

Ω -GRANDE

Ω -GRANDE

$f(n) \in \Omega(g(n))$ se $\exists c > 0$ e $\exists n_0 \geq 1$, costanti rispetto ad n , tali che valga:

$$f(n) \geq cg(n), \forall n \geq n_0$$

ESEMPI

- $f(n) = 3n + 4$ è $\Omega(n)$ perchè $f(n) > n, \forall n \geq 1$.
- $f(n) = n + 2n^2$ è $\Omega(n^2)$ perchè $f(n) > n^2, \forall n \geq 1$.
- $f(n) = c_1n + c_2$ è $\Omega(n)$ perchè $f(n) > c_1n, \forall n \geq 1$.

Θ -GRANDE

Θ -Grande

$f(n) \in \Theta(g(n))$ se:

$$f(n) \in O(g(n))$$

e anche

$$f(n) \in \Omega(g(n))$$

ESEMPI

- $f(n) = 3n + 4$ è $\Theta(n)$.
- $f(n) = n + 2n^2$ è $\Theta(n^2)$.
- $f(n) = c_1n + c_2$ è $\Theta(n)$.

o-PICCOLO

$f(n) \in o(g(n))$ se

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$$

ESEMPI

- $f(n) = 100n$ è $o(n^2)$.
- $f(n) = \frac{3n}{\log n}$ è $o(n)$.

PROPRIETA' DEGLI ORDINI

Elenchiamo di seguito alcune proprietà utili degli ordini di grandezza:

- $\max\{f(n), g(n)\} \in \Theta(f(n) + g(n)) \forall f, g : \mathbb{N} \rightarrow \mathbb{R}^+ \cup \{0\}$.
- $\sum_{i=0}^k a_i n^i \in \Theta(n^k)$ se $a_k > 0$ e $k \geq 0$.
- $\log_b n \in \Theta(\log n)$ se $b > 1$, cioè la base non è importante.
- $n^k \in o(a^n)$ se $k > 0, a > 1$.
- $(\log n)^k \in o(n^h)$ se $h, k > 0$.

ANALISI COMPLESSITA' IN PRATICA

Dato un algoritmo A e detta $t_A(n)$ la sua complessità al *caso pessimo* si cercano limiti asintotici *superiori* e/o *inferiori* per $t_A(n)$, come anticipato in [COMPLESSITA' IN TEMPO](#).

LIMITE SUPERIORE

$$t_A(n) \in O(f(n))$$

Si prova argomentando che per ogni n *"abbastanza grande"* e per ciascuna istanza di taglia n l'algoritmo esegue un numero $\leq cf(n)$ di operazioni, con c costante (che non è necessario determinare).

☰ LIMITE INFERIORE

$$t_A(n) \in \Omega(f(n))$$

Si prova argomentando che per ogni n "abbastanza grande" e per ciascuna istanza di taglia n l'algoritmo esegue un numero $\geq cf(n)$ di operazioni, con c costante (che non è necessario determinare).

In alcuni casi è comodo argomentare che per *ciascuna istanza* di *taglia* n l'algoritmo esegue almeno tale numero di operazioni.

In ogni caso, sia nel caso di *O-Grande* che di *Ω -Grande*:

- $f(n)$ deve essere più vicino possibile alla complessità vera (**tight bound**).
- $f(n)$ deve essere più semplice possibile: utilizzare solo **termini essenziali**.

TERMINOLOGIA

Nell'analisi della complessità si fa spesso riferimento alle seguenti categorie:

- **Logaritmica**: $\Theta(\log n)$ con base 2 (ma non è importante).
- **Lineare**: $\Theta(n)$.
- **Quadratica**: $\Theta(n^2)$.
- **Cubica**: $\Theta(n^3)$.
- **Polinomiale**: $\Theta(n^c)$, con $c > 0$.
- **Esponenziale**: $\Omega(a^n)$ con $a > 1$.
- **Polilogaritmica**: $\Theta((\log n)^c)$ con $c > 0$.

ESEMPIO CRITTOGRAFIA (SCELTA DELLA TAGLIA)

Nell'ambito della *crittografia a chiave pubblica* l'algoritmo più conosciuto è sicuramente **RSA** (*Rivest Shamir Adleman*).

La sua sicurezza si basa sulla difficoltà nel risolvere il seguente problema:

🔥 INTEGER FACTORIZATION

Dato un intero N prodotto di due primi p e q , determinare p e q .

Un algoritmo *banale* per risolvere il problema può essere il seguente:

Algorithm NaiveIntegerFactorization(N)

```
1: for  $p \leftarrow 2$ , to  $\lfloor \sqrt{N} \rfloor$  do
2:   if  $(N \bmod p = 0)$  then
3:     return  $\{p, N/p\}$ 
4:   end if
5: end for
```

Osservando l'algoritmo saremmo tentati di dire che la sua complessità sia $\Theta(\sqrt{N})$, tuttavia in questo caso è importante riflettere sulla *scelta della taglia dell'istanza*: dato un numero intero N servono $n = \log_2 N$ bit per rappresentarlo, quindi è più corretto scegliere n come taglia dell'istanza.

Allora troviamo che la complessità è *in realtà*:

$$\Theta(N) = \Theta(2^{n/2})$$

La complessità "vera" è **esponenziale**!

CAVEAT SULL'ANALISI ASINTOTICA AL CASO PESSIMO

A questo punto è bene chiarire alcuni punti riguardo l'analisi asintotica esposta fin'ora.

ASINTOTICAMENTE MIGLIORE

Siano A e B due algoritmi che risolvono Π e siano $t_A(n)$ e $t_B(n)$ le loro rispettive complessità.

Quando diciamo

$$t_A(n) \in o(t_B(n))$$

Intendiamo che A è **asintoticamente migliore** di B , cioè A è più efficiente di $B \forall n \geq n_0$, ma tale n_0 potrebbe potenzialmente essere *molto grande*.

COSTANTI TRASCURATE

Abbiamo visto che nell'analisi asintotica le *costanti* sono spesso trascurate, tuttavia potrebbero essere *elevate* e avere, nella pratica, un certo *impatto* sulle prestazioni.

Conviene allora dare anche una *stima delle costanti*.

ISTANZE PATOLOGICHE

Talvolta il caso pessimo potrebbe essere costituito dalle cosiddette **istanze patologiche**, mentre per tutte le **istanze di interesse** la complessità potrebbe essere asintoticamente **migliore**.

In questi casi conviene quindi:

- **Restringere il dominio** delle istanze, limitandosi a quelle di nostro interesse.
- Aggirare il caso pessimo con un'analisi **probabilistica**.
- Fare un'analisi al **caso medio**.

CORRETTEZZA

Per dimostrare la **correttezza** di un algoritmo è necessario innanzitutto provare la sua **terminazione**: bisogna assicurarsi che i cicli e l'eventuale ricorsione abbiano termine.

Più in generale, l'approccio è il seguente:

- Definire lo **stato iniziale** dell'algoritmo e quello **finale** che esso deve raggiungere.
- Decomporre l'algoritmo in **segmenti** e definire per ogni segmento lo **stato** in cui l'algoritmo si deve trovare al **termine del segmento** (**checkpoint**).
- Dimostrare che a partire dallo stato iniziale si **raggiungono in successione** gli stati specificati per la **fine di ogni segmento**.

In particolare, lo stato che deve valere alla fine dell'**ultimo segmento** deve **coincidere** (o **implicare**) lo stato **finale desiderato**.

ANALISI DI CICLI

I cicli sono i segmenti più **difficili** da analizzare perchè definiscono **implicitamente molte operazioni**, ma, insieme alla ricorsione, costituiscono gli strumenti **essenziali** per la scrittura di algoritmi/programmi interessanti.

Provare la **correttezza di un ciclo** consiste nel dimostrare che al **termine** della sua esecuzione **vale una certa proprietà** $\mathcal{L}$ che rappresenta lo stato finale del ciclo (segmento) ed è funzionale alla correttezza dell'algoritmo.

A tal fine si fa uso di un **INVARIANTE**.

INVARIANTE

Un **invariante** per un ciclo è una **proprietà** espressa in funzione delle variabili usate nel ciclo che **descrive lo stato** in cui si trova l'esecuzione alla **fine** di una generica **iterazione** del ciclo stesso.

Quindi il processo da seguire è il seguente:

CORRETTEZZA DI UN CICLO

1. Si individua un *opportuno invariante*.
2. Si dimostra che *vale all'inizio* del ciclo.
3. Si dimostra per induzione che *vale alla fine di ogni iterazione*.
4. Si dimostra che *alla fine del ciclo* l'invariante implica la proprietà $\mathcal{L}$.

RICHIAMO: INDUZIONE

Per dimostrare che una proprietà $Q(n)$ è *vera* $\forall n \geq n_0$ si procede per *induzione*:

- Si sceglie un intero $k \geq 0$.
- **BASE**: si dimostra $Q(n_0), Q(n_0 + 1), \dots, Q(n_0 + k)$.
- **PASSO INDUTTIVO**: si fissa un valore arbitrario $n \geq n_0 + k$ e si dimostra che

$$Q(m) \text{ vera } \forall m : n_0 \leq m \leq n \implies Q(n + 1) \text{ vera}$$

ESEMPIO

Prendiamo come esempio l'algoritmo *arrayMax*:

- **INPUT**: Array $A[0, \dots, n - 1]$ di $n \geq 1$ interi.
- **OUTPUT**: Massimo intero in A .

```
Algorithm arrayMax(A)
1: currMax  $\leftarrow A[0]$ 
2: for  $i \leftarrow 1$ , to  $n - 1$  do
3:   if ( $A[i] > \text{currMax}$ ) then
4:     currMax  $\leftarrow A[i]$ 
5:   end if
6: end for
7: return currMax
```

Tale algoritmo è sostanzialmente costituito da un unico segmento: il ciclo for.

- La *proprietà $\mathcal{L}$* che implica la correttezza dell'algoritmo è la seguente:

$\mathcal{L}$: Alla fine del ciclo, currMax e' il massimo intero in A

- Scegliamo a questo punto l'**invariante I**:

I = Alla fine dell'iterazione i vale $\text{currMax} = \max\{A[0], \dots, A[i]\}$

- All'*inizio* del ciclo ($i = 0$) l'invariante è **vero**: currMax è il massimo del sotto-array contenente solo $A[0]$.
- *Supponiamo* allora che l'invariante sia *vero a fine iterazione* $i < n - 1$, cioè:

$$\text{currMax} = \max\{A[0], \dots, A[i]\}$$

- e dimostriamo che vale anche alla fine dell'iterazione successiva (*Passo induttivo*). All'iterazione $i + 1$ abbiamo:

$$\text{currMax} = \max\{\text{currMax}, A[i]\} = \max\{A[0], \dots, A[i + 1]\}$$

- Quindi l'invariante *continua a valere* alla fine dell'iterazione $i + 1$.
- Alla fine dell'ultima iterazione $i = n - 1$, siccome vale l'invariante, allora currMax è *effettivamente il massimo* intero in A , che è esattamente la *proprietà* $\mathcal{L}$.

RICORSIONE

ALGORITMO RICORSIVO

E' un algoritmo che *invoca se stesso* su istanze *sempre più piccole* sfruttando la nozione di induzione.

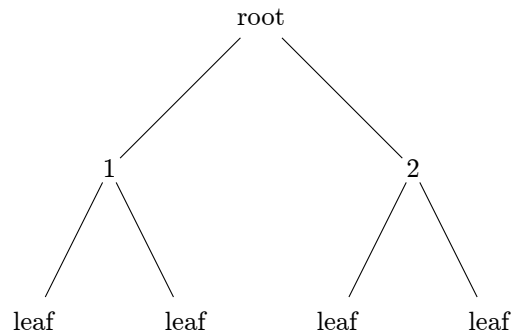
La soluzione di una istanza di taglia n è ottenuta:

- Direttamente: se $n = n_0, n_0 + 1, \dots, n_0 + k$ (casi base).
- Riconducendosi alla soluzione di $r \geq 1$ istanze di taglia $< n$: se $n > n_0 + k$.
Se $r = 1$ si parla di *linear recursion*.

ESECUZIONE DI UN ALGORITMO RICORSIVO

All'esecuzione di un algoritmo ricorsivo su una data istanza è associato un **albero della ricorsione** tale che:

- Ogni **nodo** corrisponde a un'*invocazione ricorsiva distinta* fatta durante l'esecuzione dell'algoritmo.
- La **radice** dell'albero corrisponde alla prima invocazione e i **figli** di un nodo x sono associati alle invocazioni ricorsive fatte direttamente dall'invocazione corrispondente a x .
- Le **foglie** (nodi senza figli) rappresentano i *casi base*.



🔥 NOTA

L'albero della ricorsione è detto anche **recursion trace**.

Nell'esecuzione di un programma (in Java per esempio) entrano in gioco due spazi di memoria:

1. **HEAP**: memoria per allocazione dinamica, destinata a contenere gli oggetti.
2. **STACK**: spazio destinato a memorizzare variabili locali ai metodi e i riferimenti agli oggetti.
 - Per ogni invocazione di un metodo viene inserito un *record di attivazione* contenente variabili e riferimenti a oggetti relativi a *quella invocazione*.
 - Un *RA* viene *eliminato* quando l'*invocazione* corrispondente *termina*.
 - I *RA* sono inseriti/eliminati con politica *LIFO* e in ogni istante si può accedere ai dati relativi all'ultimo RA ma non agli altri.

COMPLESSITA' DI ALGORITMI RICORSIVI

La complessità di un algoritmo ricorsivo A può essere *stimata* grazie all'*albero della ricorsione*.

Consideriamo l'albero associato all'esecuzione di A su una istanza i di taglia n :

- A ogni *nodo* è attribuito un **costo** pari al numero di operazioni eseguite dall'invocazione corrispondente a quel nodo, *escluse* quelle fatte dalle *invocazioni ricorsive al suo interno*.
- Il numero totale di operazioni eseguite da A per risolvere i si ottiene sommando i costi associati a tutti i nodi.

Per ottenere un *upper bound* a $t_A(n)$, si ricava una stima per eccesso del numero totale di operazioni che valga per tutte le istanze i di taglia n .

Per ottenere un *lower bound* si trova un'istanza particolare o si fa una stima inferiore che va bene per tutte le istanze.

Vediamo come applicare questo procedimento all'algoritmo *reverseArray(A,i,j)*:

- **INPUT**: Array A , indici $i, j \geq 0$.
- **OUTPUT**: Array A con gli elementi in $A[i \div j]$ ribaltati.

```
Algorithm reverseArray(A,i,j)
1: if ( $i < j$ ) then
2:   exchange  $A[i]$  with  $A[j]$ 
3:   REVERSEARRAY( $A, i + 1, j - 1$ )
4: end if
5: return
```

Notiamo che l'algoritmo esegue *al più una chiamata ricorsiva* per ogni istanza: si tratta infatti di una *ricorsione lineare*.

Albero della ricorsione associato ad una generica istanza di taglia $n (= j - i + 1)$:

- **Numero di livelli**: $\lfloor \frac{n}{2} \rfloor + 1$ dato che ad ogni chiamata successiva riduco di 2 la taglia.
- **Costo di ciascun nodo**: $\Theta(1)$ operazioni.

Per cui ogni istanza di *taglia* n richiede $\Theta(n)$ operazioni.

CORRETTEZZA DEGLI ALGORITMI RICORSIVI

Per provare la **correttezza** o una qualsiasi altra proprietà di un algoritmo ricorsivo A si ricorre ancora una volta all'**induzione**.

Sia n la taglia dell'istanza:

- Si dimostra la correttezza per i *casi base* $n \in [n_0, n_0 + k]$.
- Supponendo che A risolva correttamente tutte le istanze di taglia $m \in [n_0, n]$ per un qualche $n > n_0 + k$ si dimostra che esso *risolve correttamente anche* tutte le istanze di *taglia* $n + 1$.

Vediamo come applicare il procedimento all'algoritmo *linearSum(A,n)*.

- **INPUT**: Array A di interi, indice $n \geq 1$.
- **OUTPUT**: $\sum_{i=0}^{n-1} A[i]$.

```
Algorithm linearSum(A,n)
1: if ( $n = 1$ ) then
2:   return  $A[0]$ 
3: end if
4: return LINEARSUM( $A, n - 1$ ) +  $A[n-1]$ 
```

- **CASO BASE:** $n = 1$, la correttezza è banale.
- **PASSO INDUTTIVO:** Fissiamo $n \geq 1$ arbitrario e assumiamo che $\text{linearSum}(A, j)$ sia *corretto* $\forall A, \forall 1 \leq j \leq n$.
Consideriamo ora $\text{linearSum}(A, n + 1)$ con A array di $n + 1$ elementi, che restituirà $\text{linearSum}(A, n) + A[n]$.
Ma questo è uguale a $\left(\sum_{i=0}^{n-1} A[i]\right) + A[n]$ per ipotesi induttiva.
Ma tale valore è anche $\sum_{i=0}^n A[i]$.
- Concludiamo quindi che il *valore restituito* è **corretto**.

CALCOLO EFFICIENTE DI POTENZE

Consideriamo il seguente problema:

🔥 PROBLEMA

Dato $x \in \mathbb{R}$ e $n \geq 0$ intero, calcolare $p(x, n) = x^n$.

Il primo algoritmo che ci viene in mente consiste nel moltiplicare x per sè stesso n volte, la cui complessità è chiaramente $\Theta(n)$.

Vediamo tuttavia di seguito un approccio più efficiente.

🔥 OSSERVAZIONE CHIAVE:

$$p(x, n) = \begin{cases} 1 & n = 0 \\ x \cdot p(x, \frac{n-1}{2})^2 & n > 0 \text{ dispari} \\ p(x, \frac{n}{2})^2 & n > 0 \text{ pari} \end{cases}$$

Vediamo ora l'algoritmo *power(x,n)*:

- **INPUT:** $x \in \mathbb{R}$ e $n \geq 0$ intero.
- **OUTPUT:** $p(x, n)$.

```

Algorithm power( $x, n$ )
1: if ( $n = 0$ ) then
2:   return 1
3: end if
4: if ( $n$  even) then
5:    $y \leftarrow \text{POWER}(x, n/2)$ 
6:   return  $y \cdot y$ 
7: else
8:    $y \leftarrow \text{POWER}(x, (n - 1)/2)$ 
9:   return  $x \cdot y \cdot y$ 
  
```

Consideriamo n stesso come taglia dell'istanza e supponiamo $n \geq 1$:

- Alla i -esima chiamata ricorsiva l'algoritmo viene invocato per un esponente $n_i \leq \frac{n}{2^i}$. Di conseguenza, l'albero della ricorsione avrà $O(\log n)$ **nodi**.
- Ogni invocazione di `power` esegue $\Theta(1)$ operazioni, esclusa l'eventuale chiamata ricorsiva. Quindi il **costo per nodo** è $\Theta(1)$.
- Concludiamo che $t_{\text{power}}(n) \in O(\log n)$.

APPLICAZIONE AI NUMERI DI FIBONACCI

Fissati il **rapporto aureo** $\phi = \frac{1+\sqrt{5}}{2}$ e $\hat{\phi} = \frac{1-\sqrt{5}}{2}$ possiamo calcolare l' *n -esimo numero di Fibonacci* grazie alla seguente relazione:

$$F(n) = \frac{1}{\sqrt{5}}(\phi^n - \hat{\phi}^n)$$

Possiamo allora scrivere l'algoritmo *powerFib(n)*:

- **INPUT**: intero $n \geq 0$.
- **OUTPUT**: $F(n)$.

Algorithm powerFib(n)

```

1:  $\phi \leftarrow \frac{1+\sqrt{5}}{2}$ 
2:  $\hat{\phi} \leftarrow \frac{1-\sqrt{5}}{2}$ 
3: return POWER( $\phi, n$ ) - POWER( $\hat{\phi}, n$ )

```

Da quanto provato per `power`, otteniamo che la complessità di `powerFib` è $O(\log n)$.